Shor's Algoritm and Related Topics

Carl Fredrik Nordbø Knutsen,[1] Carl Peter Kirkebø,[2] Emma Storberg,[1] and Jonas Båtnes[1]

[1] *University of Oslo, Department of Physics*

[2] *University of Oslo, Department of Informatics*

(Dated: May 28, 2025)

In this project, we investigate fundamental algorithms of quantum computing, focusing on the Quantum Fourier Transform (QFT), Quantum Phase Estimation (QPE), and Shor's algorithm for integer factorization. We present independent implementations of these algorithms, developed without external quantum computing libraries, alongside parallel implementations using Qiskit [7] to benchmark correctness and efficiency. We outline the core mathematical principles behind each algorithm, construct and validate quantum circuits, and analyze performance through small-scale examples. We also discuss possible challenges inherent in the probabilistic nature of Shor's algorithm. Our QFT and QPE methods are validated against their built-in Qiskit counterparts, and using our implementation of Shor's algorithm, we successfully factorize the integers 15 and 21.

I. Introduction

Quantum technology is a much researched and discussed topic today for its ability to utilize quantum principles to allow for computation and processing that we are otherwise unable to do classically. One of the proposed use cases is in the field of cryptography, where the potential of breaking the security of the widely-used Rivest-Shamir-Adleman (RSA) algorithm is of particular interest. Among other things, algorithms like RSA, based on the hardness of certain mathematical problems, famously provide security across much of the internet [2]. Specifically, RSA takes advantage of the difficulty of factoring large composite numbers, which is generally considered computationally infeasible for classical computers in any reasonable amount of time. This problem is also well suited for information security purposes because while factorizing is hard, verifying a candidate solution (i.e. multiplying two integers) is easy.

However, the advent of quantum computing poses significant challenges to traditional cryptographic methods, as exemplified by *Shor's algorithm*. Developed by mathematician Peter Shor in 1994 [11], this quantum algorithm can efficiently factor large integers exponentially faster than current algorithms do, thus threatening the security afforded by RSA. As quantum technology continues to evolve, understanding and implementing Shor's algorithm becomes paramount for assessing the vulnerability of current cryptographic protocols and preparing for a future where quantum computers may become practical.

In this paper, we undertake a systematic step-by-step implementation of Shor's algorithm. Our approach begins with the *Quantum Fourier Transform* (QFT), a fundamental quantum operation. Next, we implement *Quantum Phase Estimation* (QPE), which incorporates the inverse QFT as a subroutine. Finally, we integrate QPE into the complete execution of Shor's algorithm, illustrating the roles of these components in achieving efficient integer factorization.

Throughout the paper, we emphasize testing and validation of our implementation at each stage using the quantum simulation package Qiskit [7]. By providing detailed explanations and the results of our implementations, we aim to contribute to the growing body of work in quantum computing education and its applications in cryptography. The structure of the paper is as follows: First, we will introduce the QFT, followed by QPE, and finally, we will present Shor's algorithm as a holistic application of the prior components. In particular, our aim is to be able to use our implementation of Shor's algorithm to factorize the integers 15 and 21. Through this process, we aim to shed light on the intricacies of these quantum algorithms and the implications they bear for the future of cryptography. All code used to obtain results discussed in this paper can be found on our GitHub: https://github.com/carlfre/fys5419-project-2.

II. Methods

The three quantum algorithms we are working with in this paper build on each other. First, we set up the QFT, the quantum analog to the *Discrete Fourier Transform* (DFT). Next, we use QFT in the QPE algorithm, which is itself used in the final algorithm we are considering, namely Shor's algorithm for factorization of integers.

We will begin by providing some background on the DFT before delving into the quantum algorithms in question. In general, the *Fourier Transform* is a function mapping a signal to a decomposition of its constituent frequencies. This can be done as an integral, representing a continuous frequency spectrum, or, as in our case, discretely. In the classical case, the DFT maps a vector $(x_0, x_1, \ldots, x_{M-1}) \in \mathbb{C}^M$ to a vector $(y_0, y_1, \ldots, y_{M-1}) \in$

$\mathbb{C}^M$. The definition of the DFT is shown below:

$$y_k = \frac{1}{\sqrt{M}} \sum_{j=0}^{M-1} x_j e^{-\frac{2\pi i}{M} jk}, \quad k = 0, 1, \dots M-1$$

To compute the DFT directly from this expression typically requires a double for-loop: an outer loop over k, representing the discrete frequencies we wish to consider, and an inner loop over j up to $M-1$, which is the length of the signal in question. As a result, the asymptotic runtime of this method of calculation, known as *DFT by correlation*, is on the order of M^2.

The *Fast Fourier Transform* (FFT) is a widely used implementation of the DFT, which offers a significant runtime improvement over evaluating the DFT by correlation, down from $\mathcal{O}(M^2)$ to $\mathcal{O}(M \log M)$ floating point operations (FLOPs). Besides the benefits in terms of raw speed, the FFT also offers an advantage in terms of precision, since fewer operations mean less round-off errors. The FFT has been described as "absolutely critical" for processing in cases where M is large [12]; it has countless applications across a variety of fields, with MIT professor Gilbert Strang calling it "the most important numerical algorithm of our time" [10]. Still, as we will see, there are instances in which the speedup afforded by the FFT does not suffice for the task at hand.

A. Quantum Fourier Transform

The QFT is another implementation of the DFT which utilizes quantum principles to achieve an estimated asymptotic runtime exponentially better than even that of the FFT – from $\mathcal{O}(M \log M)$ FLOPs to $\mathcal{O}((\log M)^2)$ quantum gates [16]. We are able to compare these metrics because we consider FLOPs and the application of quantum gates to be constant operations, i.e. in $\mathcal{O}(1)$ time with respect to input size M. Mathematically, the QFT and the DFT have the same form, but the notation is typically different. The QFT is generally computed on a set of orthonormal basis state vectors $|0\rangle, |1\rangle, ..., |M-1\rangle$. We can describe the transform as a linear operator which performs the following action on the basis states:

$$|j\rangle \mapsto \frac{1}{\sqrt{M}} \sum_{k=0}^{M-1} e^{\frac{2\pi i}{M} jk} |k\rangle.$$

Note that conventions for the sign in the exponent vary, so in our case, applying the QFT has the same effect as the inverse DFT, and vice versa.

For an arbitrary state that is a linear combination of basis states, we can write the QFT as:

$$\sum_{j=0}^{M-1} x_j |j\rangle \mapsto \sum_{k=0}^{M-1} y_k |k\rangle,$$

where the amplitudes y_k are DFTs of the x_j.

When working with quantum systems, we require that all transformations are unitary, meaning $\mathbb{F}\mathbb{F}^\dagger = \mathbb{1}$. This also applies to the QTF, which we demonstrate below. First, we write the QFT as a matrix $\mathbb{F}$, along with its adjoint $\mathbb{F}^\dagger$:

$$\mathbb{F} = \frac{1}{\sqrt{M}} \sum_{j,k=0}^{M-1} e^{\frac{2\pi i}{M} jk} |k\rangle \langle j|$$

$$\mathbb{F}^\dagger = \frac{1}{\sqrt{M}} \sum_{j,k=0}^{M-1} e^{-\frac{2\pi i}{M} jk} |j\rangle \langle k|$$

Inserting these definitions gives:

$$\mathbb{F}\mathbb{F}^\dagger = \frac{1}{M} \sum_{j,k,j',k'=0}^{M-1} e^{\frac{2\pi i}{M}(jk - j'k')} |k\rangle \langle j| \, |j'\rangle \langle k'|$$

We can use that $\langle j|j'\rangle = \delta_{jj'}$ to effectively only consider a single index j. Splitting up the sum notation helps us see this more clearly:

$$\mathbb{F}\mathbb{F}^\dagger = \frac{1}{M} \sum_{k,k'=0}^{M-1} \sum_{j=0}^{M-1} e^{\frac{2\pi i}{M} j(k-k')} |k\rangle \langle k'|$$

At this point, and also in later applications, we need the following important identity:

$$\frac{1}{M} \sum_{k=1}^{M} e^{\frac{2\pi i}{M} k(n-m)} = \begin{cases} 1 & \text{if } n \equiv m \pmod{M} \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

We will now show why eq. (1) is true. For brevity, we use ω to denote a principal M-th root of unity, in this case $e^{\frac{2\pi i}{M}}$. Ignoring the factor $1/M$, the sum can be written as the geometric series $1 + \omega^{n-m} + \omega^{2(n-m)} + \cdots + \omega^{(M-1)(n-m)}$. For $z \neq 1$, we have the well-known identity

$$1 + z + z^2 + \cdots + z^{M-1} = \frac{z^M - 1}{z - 1}. \tag{2}$$

Observe that $\omega^{n-m} = 1$ if and only if $n \equiv m \pmod{M}$. In this case, all summands equal 1 and their average becomes 1 as desired. If $n \not\equiv m \pmod{M}$, then $\omega^{n-m} \neq 1$ but $(\omega^{n-m})^M = 1$. Plugging $z = \omega^{n-m}$ into the right hand side of eq. (2) gives that the sum is zero. In appendix A, we also provide a "geometric proof" that perhaps brings out the intuition more clearly.

Hence, we have now determined that we can replace the sum over j by the Kronecker delta $\delta_{kk'}$. As before, doing so lets us only consider a single index k:

$$\mathbb{F}\mathbb{F}^\dagger = \sum_{k=0}^{M-1} |k\rangle \langle k| = \mathbb{I}_M.$$

Evidently, we obtain the identity matrix of dimension $M \times M$, which gives us the desired result that $\mathbb{F}$ is unitary. We display the QFT as a matrix for 1-, 2- and 3-qubit systems below, denoted $\mathbb{F}_2$, $\mathbb{F}_4$ and $\mathbb{F}_8$ respectively (corresponding to the value of M in the systems).

$$\mathbb{F}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & \omega\end{bmatrix}, \qquad \mathbb{F}_4 = \frac{1}{\sqrt{4}}\begin{bmatrix}1 & 1 & 1 & 1\\ 1 & \omega & \omega^2 & \omega^3\\ 1 & \omega^2 & \omega^4 & \omega^6\\ 1 & \omega^3 & \omega^6 & \omega^9\end{bmatrix},$$

$$\mathbb{F}_8 = \frac{1}{\sqrt{8}}\begin{bmatrix}1 & 1 & 1 & 1 & 1 & 1 & 1 & 1\\ 1 & \omega & \omega^2 & \omega^3 & \omega^4 & \omega^5 & \omega^6 & \omega^7\\ 1 & \omega^2 & \omega^4 & \omega^6 & \omega^8 & \omega^{10} & \omega^{12} & \omega^{14}\\ 1 & \omega^3 & \omega^6 & \omega^9 & \omega^{12} & \omega^{15} & \omega^{18} & \omega^{21}\\ 1 & \omega^4 & \omega^8 & \omega^{12} & \omega^{16} & \omega^{20} & \omega^{24} & \omega^{28}\\ 1 & \omega^5 & \omega^{10} & \omega^{15} & \omega^{20} & \omega^{25} & \omega^{30} & \omega^{35}\\ 1 & \omega^6 & \omega^{12} & \omega^{18} & \omega^{24} & \omega^{30} & \omega^{36} & \omega^{42}\\ 1 & \omega^7 & \omega^{14} & \omega^{21} & \omega^{28} & \omega^{35} & \omega^{42} & \omega^{49}\end{bmatrix}$$

To further illustrate how processing with these matrices might work, let us consider the n-qubit initial state $|00\ldots0\rangle$ and see what happens. This vector only has a nonzero value (of 1) in its very first entry. Therefore, when applying one of these matrices, we extract only the values in the first column and scale by our global constant of $\frac{1}{\sqrt{M}}$. It is clear that we obtain the vector $\frac{1}{\sqrt{M}}\begin{bmatrix}1 & 1\ldots & 1\end{bmatrix}^T$, which has length 1 and an equal component in every entry.

We confirm this result by considering the linear operator

$$|j\rangle \mapsto \frac{1}{\sqrt{M}}\sum_{k=0}^{M-1} e^{\frac{2\pi i}{M}jk}|k\rangle.$$

With the lexicographical ordering of basis states we are working with, the initial state $|00\ldots0\rangle$ can simply be written as $|0\rangle$. Thus, when processing, we set $j = 0$ in the exponent, meaning each term $|k\rangle$ in the sum is scaled by $e^0 = 1$, followed by the factor $\frac{1}{\sqrt{M}}$ outside of the sum. In other words, we create a state from the sum of M basis state vectors $|k\rangle$, each scaled by $\frac{1}{\sqrt{M}}$. This corresponds to the result when considering the matrix multiplication.

$$\frac{1}{\sqrt{M}}\sum_{k=0}^{M-1}|k\rangle = \frac{1}{\sqrt{M}}\left(\begin{bmatrix}1\\0\\\vdots\\0\end{bmatrix} + \cdots + \begin{bmatrix}0\\0\\\vdots\\1\end{bmatrix}\right) = \frac{1}{\sqrt{M}}\begin{bmatrix}1\\1\\\vdots\\1\end{bmatrix}$$

In our own code, we defined the QFT not with its mathematical expression or the matrices we determined, but rather through circuitry. The circuit we used is shown in figure 1, where the gates labeled R_k represent controlled-$U1$ gates, which apply a phase shift to the target if the controlled qubit is 1. They are defined as:

$$R_k = \begin{bmatrix}1 & 0\\ 0 & e^{\frac{2\pi i}{2^k}}\end{bmatrix}.$$

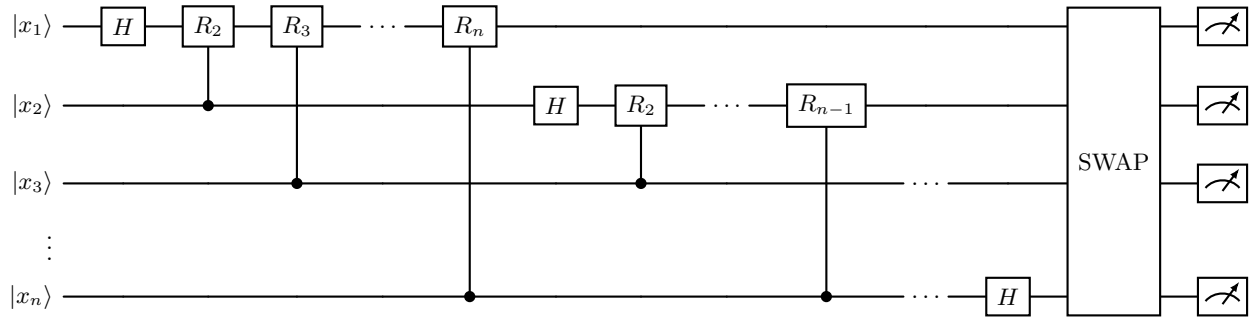

FIG. 1. QFT circuit: the input state $|x_1\ldots x_n\rangle$ is processed through a series of Hadamard and controlled phase rotation gates. Each qubit undergoes a Hadamard gate followed by controlled-R_k rotations, entangling it with all subsequent qubits. A final SWAP gate reverses the order of the qubits to match the correct output basis of the QFT.

In short, the circuit takes a number in its binary representation as input. For each qubit, starting from the one representing the least significant digit, we apply a Hadamard gate (H) to create a superposition, and then we use the R_k gates to rotate the qubit some amount, depending on which qubit in the sequence it is and whether the value of the control is 1. This process is repeated until we reach the last qubit, representing the most significant digit, where only the H gate is applied. Since the QFT produces the Fourier transform of the input in the wrong order, we reverse the order of the qubits after all other processing is applied.

Let us take a moment to convince ourselves that this circuit does indeed reflect the same operation as the expression we worked with before. Recall that the QFT on n qubits is defined by its action on computational basis states, with $M = 2^n$:

$$\mathrm{QFT}\,|j\rangle = \frac{1}{\sqrt{M}} \sum_{y=0}^{M-1} e^{\frac{2\pi i}{M} jk}\,|k\rangle\,.$$

Let $|x\rangle = |x_0 x_1 \dots x_{n-1}\rangle$, where x_0 is the most significant bit and x_{n-1} is the least significant bit. We have the following binary representation:

$$0.x_1 x_2 \dots x_k = \frac{x_1}{2} + \frac{x_2}{4} + \cdots + \frac{x_k}{2^k}.$$

The QFT circuit acts on input $|x\rangle$ by placing the first qubit (most significant bit) in the superposition with H, and thereafter the phase of all the other qubits x_i control whether a rotation is applied to it:

$$\frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i 0.x_1 x_2 \dots x_{n-1}}|1\rangle\right),$$

Following the same steps of processing, with one fewer rotation gate applied, the second qubit becomes:

$$\frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i 0.x_2 x_3 \dots x_{n-1}}|1\rangle\right).$$

This continues until the final (least significant) qubit is transformed into:

$$\frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i 0.x_{n-1}}|1\rangle\right).$$

Therefore, the output state of the full QFT circuit (before qubit reversal) is:

$$\frac{1}{\sqrt{2^n}} \bigotimes_{k=0}^{n-1} \left(|0\rangle + e^{2\pi i 0.x_{k+1} x_{k+2} \dots x_{n-1}}|1\rangle\right).$$

$$\frac{1}{\sqrt{M}}\left(|0\rangle + e^{2\pi i 0.x_{n-1}}|1\rangle\right) \otimes \cdots \otimes \left(|0\rangle + e^{2\pi i 0.x_1 x_2 \dots x_{n-1}}|1\rangle\right),$$

Here, the bits are reversed due to the ordering of the qubits in the circuit. This is typically corrected at the end by swap gates, like we have chosen to do. To see why this product form is equivalent to the original QFT definition, note that expanding the tensor product amounts to summing over all possible bit strings $|k_0 k_1 \dots k_{n-1}\rangle$ for the output register. Each bit k_m picks either the $|0\rangle$ or $|1\rangle$ component from the corresponding qubit's superposition. When the $|1\rangle$ component is chosen, it contributes a phase factor of $\exp(2\pi i 0.x_{m+1} x_{m+2} \dots x_{n-1})$.

The total phase for any computational basis state $|k\rangle$ in the output is thus the product of the phases contributed by each qubit where $k_m = 1$, which can be rewritten as a sum in the exponent:

$$\exp(2\pi i \sum_{m=0}^{n-1} k_m \cdot 0.x_{m+1} x_{m+2} \dots x_{n-1}).$$

The sum in the expression encodes the binary fractional expansion of the input bits x_l weighted by the output bits k_m. When evaluated, it turns out that this sum equals the fraction $\frac{jk}{M}$, where j is the integer represented by the input bits $\{x_l\}$ and k is the integer represented by the output bits $\{k_m\}$. In other words, each output basis state $|k\rangle$ in the expansion acquires the phase $e^{2\pi i \frac{jk}{M}}$, exactly as in the original QFT definition. Thus, the QFT circuit's stepwise application of H and R_k gates builds up the same global phase pattern that the QFT unitary applies in one step, confirming that the circuit implements the QFT operation as defined.

For the purposes of this project, we are primarily interested in the QFT because we need its inverse (IQFT) for the next algorithm, QPE. In terms of circuitry, IQFT corresponds to running the QFT circuit in the opposite direction, with the inverse of each gate applied. The Hadamard and SWAP operations are their own inverses, and the inverse of the rotation gate R_k is given by:

$$R_k^\dagger = \begin{bmatrix} 1 & 0 \\ 0 & e^{-\frac{2\pi i}{2^k}} \end{bmatrix}.$$

The main intuition about the QFT that we would like to highlight before moving on is this: The classical DFT is best understood as a way of decomposing signals into their components, and this intuition holds when considering the QFT as well. More generally however, we can simply consider the QFT a way of mapping from the computational basis $|0\rangle, |1\rangle$ to the Fourier basis $|+\rangle, |-\rangle$. This is often naturally interpreted as a mapping to the frequency domain.

B. Quantum Phase Estimation

QPE aims to estimate the value θ associated with an eigenvalue $e^{2\pi i\theta}$ of a unitary operator U. Before diving into how exactly we do this, we will familiarize ourselves with the concept of *phase kickback*, referring to the transferal of phase from one qubit to another, as described by the following relation [1]:

$$|0\rangle \otimes |\psi\rangle + |1\rangle \otimes e^{2\pi i\theta}|\psi\rangle = \left(|0\rangle + e^{2\pi i\theta}|1\rangle\right) \otimes |\psi\rangle \quad (3)$$

Essentially, QPE works by writing the phase of U (in the Fourier basis) to n qubits in a counting register, and then applying the IQFT to translate this from the Fourier basis into the computational basis, which is possible for us to measure [1]. The circuit is shown in figure 2.

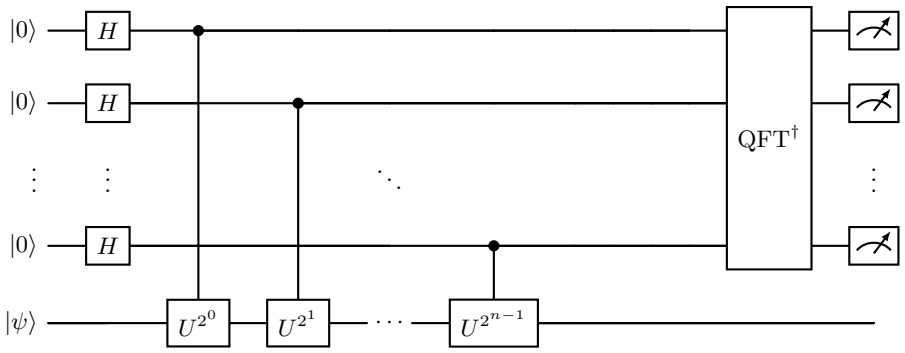

FIG. 2. QPE circuit: the top n qubits are initialized in the state $|0\rangle$ and undergo Hadamard gates, followed by controlled powers of U acting on the eigenstate $|\psi\rangle$. An inverse QFT is then applied before measurement.

First, we set up the registers we need. We have $|\psi\rangle$ representing the eigenvector associated with the eigenvalue $e^{2\pi i\theta}$, and we need a counting register of n qubits where we will store the value of θ:

$$|\psi_0\rangle = |0\rangle^{\otimes n}|\psi\rangle$$

We apply an n-bit Hadamard gate to the counting register to create a superposition before further processing is applied:

$$|\psi_1\rangle = \frac{1}{\sqrt{2^n}}\left(|0\rangle + |1\rangle\right)^{\otimes n}|\psi\rangle$$

At this point, we introduce the controlled unitary CU, which applies the operator U on the target register only when the control qubit is 1. The result of the operation $U^{2^j}|\psi\rangle$ can be written as:

$$U^{2^j}|\psi\rangle = U^{2^j-1}U|\psi\rangle = U^{2^j-1}e^{2\pi i\theta}|\psi\rangle = \cdots = e^{2\pi i 2^j\theta}|\psi\rangle$$

Now, we apply all the n controlled operations CU^{2^j} for $j \in \{0, 1, \ldots, n-1\}$. Because of the phase kickback relation shown in eq. (3), we now have the state:

$$\begin{aligned}|\psi_2\rangle &= \frac{1}{\sqrt{2^n}}\left(|0\rangle + e^{2\pi i\theta 2^{n-1}}|1\rangle\right)\otimes\cdots\otimes\left(|0\rangle + e^{2\pi i\theta 2^1}|1\rangle\right)\\ &\quad\otimes\left(|0\rangle + e^{2\pi i\theta 2^0}|1\rangle\right)\otimes|\psi\rangle\\ &= \frac{1}{\sqrt{2^n}}\sum_{k=0}^{2^n-1} e^{2\pi i\theta k}|k\rangle\otimes|\psi\rangle\end{aligned}$$

where k denotes the integer representation of n-bit binary numbers. We see now that this is the resulting expression when the QFT is applied. Thus, using our intuition about the QFT and change of basis, we see that IQFT can be applied here to write the value we have now determined in the Fourier basis in terms of the standard computational basis instead.

C. Shor's Algorithm

We have now built the foundation we need to fully implement Shor's algorithm for factorization. We will denote the number to be factorized by N. As the algorithm is considered a *hybrid* algorithm (consisting of both a classical and quantum part), it is natural to split our analysis and explanation into two sections for the number theoretic and quantum computational parts of the algorithm respectively.

Number Theoretic Part

This part relies to a great extent on the existence of an integer a relative prime to N that has *even order* modulo N. If this is satisfied, we have that

$$N \mid \left(a^{r/2} - 1\right)\cdot\left(a^{r/2} + 1\right)$$

Then $\gcd(N, a^{r/2} - 1)$ and $\gcd(N, a^{r/2} + 1)$ should be good candidates for producing factors of N.

The elements a that are coprime to N form an abelian group; the multiplicative group of integers modulo N [13]. In the following it will be denoted by $U(\mathbb{Z}/N\mathbb{Z})$.

Since there are $\varphi(N)$ integers between 1 and N that are coprime to N, the order of the group is $\varphi(N)$. By Lagrange's theorem [15], any element raised to the power of the group's order equals the identity. In number theory, this result is known as Euler's theorem, which states that if $\gcd(a, N) = 1$, then

$$a^{\varphi(N)} \equiv 1 \pmod{N}.$$

Moreover, it follows from Lagrange's theorem—or from elementary number theory—that the order r of any such element a divides $\varphi(N)$.

There is a priori no guarantee that the order r is even. However, it turns out that in our case most elements will be of even order. The argument is as follows: consider a general finite abelian group G of order $n > 1$ and let the unique factorization of n into distinct prime powers be $n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k}$. Then $G \cong A_1 \times A_2 \times \cdots \times A_k$ where $|A_i| = p_i^{\alpha_i}$. See e.g. [3] Theorem 5, page 161.

It follows from this that if $|G| = 2^m \cdot Q$, where Q is odd, then $G \cong H \times K$ where $|H| = 2^m$ and $|K| = Q$. Since the order of an element in a group divides the order of the group, the order of all elements in H will be be of the form 2^t including the case $t = 0$, whereas the order of all elements in K will be odd. The elements in G of odd order thus correspond one to one to the elements (e, k). Hence, Q is the number of elements of odd order in G.

Let us now consider the case $N = pq$ where p and q are distinct odd primes. We have that $|U(\mathbb{Z}/N\mathbb{Z})| = \varphi(N) = (p-1)(q-1)$. The factors $p-1$ and $q-1$ are both even and admit factorizations $p - 1 = 2^{m_p} \cdot \beta_p$ and $q - 1 = 2^{m_q} \cdot \beta_q$ where $m_p, m_q \geq 1$, and both β_p and β_q are odd. The number of odd elements in $U(\mathbb{Z}/N\mathbb{Z})$ will be $\beta_p \cdot \beta_q$, and the proportion of elements of even order will therefore be

$$1 - \frac{\beta_p \cdot \beta_q}{2^{m_p+m_q} \cdot \beta_p \cdot \beta_q} = 1 - \frac{1}{2^{m_p+m_q}}$$

The point is of course that in advance, we do not know p or q. Thus, we only know that $m_p, m_q \geq 1$. It follows that at most $\left(\frac{1}{2}\right)^2 = \frac{1}{4}$ of the elements are of odd order, meaning at least $\frac{3}{4}$ of the elements will be of even order.

Example: Consider the case $N = 35$ which factors as $N = pq$ with $p = 5$ and $q = 7$. In this case, we have $p - 1 = 2^2 \cdot 1$ and $q - 1 = 2^1 \cdot 3$. Thus, we see that $m_p = 2$, $\beta_p = 1$, $m_q = 1$ and $\beta_q = 3$, meaning there will be 3 elements of odd order in $U(\mathbb{Z}/35\mathbb{Z})$ and 21 elements of even order. This is illustrated in table I, where the odd orders are highlighted in yellow.

a	1	2	3	4	6	8	9	11	12	13	16	17
order	1	12	12	6	2	4	6	3	12	4	3	12

a	18	19	22	23	24	26	27	29	31	32	33	34
order	12	6	4	12	6	6	4	2	6	12	12	2

TABLE I. Order of elements in $U(\mathbb{Z}/35\mathbb{Z})$

We pick a random a of even order, for instance $a = 4$. Then $\gcd(4^{6/2} - 1, 35) = 7$ and $\gcd(4^{6/2} + 1, 35) = 5$, and we have recovered the prime factors of N. However, we might have made an unlucky choice. For $a \in \{19, 24, 34\}$, one of the gcds would equal 1 and the other would equal 35, as we see that trivially $\gcd(34^{2/2} - 1, 35) = 1$ and $\gcd(34^{2/2} + 1, 35) = 35$.

Quantum Computing Part

As described in subsection II A, the Fourier Transform describes the extent to which various frequencies are present in the input. We will give a sketch of the most important steps; for details, refer to Hundt [5].

Step 1: Choose a random a where $1 < a < N$. If $\gcd(a, N) > 1$ we have found a prime factor of N and the algorithm terminates. If not, $\gcd(a, N) = 1$ and we move to step 2

Step 2: Define the gate $U_{a,N}$ by

$$U_{a,N} |x\rangle = |xa \bmod N\rangle$$

For ease of notation, let $U := U_{a,N}$. The order of a will be captured by the periodicity of U:

$$\begin{aligned}
U^0 |1\rangle &= |1 \bmod N\rangle \\
U^1 |1\rangle &= |a \bmod N\rangle \\
U^2 |1\rangle &= \left|a^2 \bmod N\right\rangle \\
&\vdots \\
U^r |1\rangle &= |a^r \bmod N\rangle = |1 \bmod N\rangle \\
&\vdots
\end{aligned}$$

Step 3: Let $\omega = e^{\frac{2\pi i}{r}}$ and define the sum

$$|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} \omega^{-sk} \left|a^k \bmod N\right\rangle \tag{4}$$

Consider U applied to eq. (4). This gives

$$\begin{aligned}
U |u_s\rangle &= \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} \omega^{-sk} U \left|a^k \bmod N\right\rangle \\
&= \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} \omega^{-sk} \left|a^{k+1} \bmod N\right\rangle \\
&= \omega^s \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} \omega^{-s(k+1)} \left|a^{k+1} \bmod N\right\rangle \\
&= \omega^s |u_s\rangle .
\end{aligned}$$

In the the last equality, we used the periodicity of $a^k \bmod N$. Thus, we have shown that $|u_s\rangle$ is an eigenvector of U with eigenvalue ω^s. Now, the crucial observation is that

$$\begin{aligned}
\frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |u_s\rangle &= \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} \left(\frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} \omega^{-sk} \left|a^k \bmod N\right\rangle \right) \\
&= \frac{1}{r} \sum_{k=0}^{r-1} \left(\sum_{s=0}^{r-1} \omega^{-sk} \left|a^k \bmod N\right\rangle \right) .
\end{aligned}$$

The inner sum $\sum_{s=0}^{r-1}\omega^{-sk}$ is geometric, and it follows from eq. (1) that it equals r if $k \equiv 0 \pmod r$ and 0 otherwise. Hence,

$$\frac{1}{\sqrt{r}}\sum_{s=0}^{r-1}|u_s\rangle = |1 \bmod N\rangle\,. \tag{5}$$

Eq. (5) combined with the eigenvalue of $|u_s\rangle$ is in many ways the crux of Shor's algorithm, as we will see in the next step.

Step 4: Set up the appropriate circuit and use the QPE to estimate the phase. The circuit will estimate the eigenvalue $\omega^s = e^{\frac{2\pi i s}{r}}$ for one of the $0 \le s \le r-1$, and the QPE will recover an estimate for the fraction $\frac{s}{r}$.

Step 5: From step 4, there has been obtained a binary number that approximates $\frac{s}{r}$. To try to recover r, we use the *continued fractions* technique [14]. This algorithm will return the value of the fraction $\frac{s}{r}$ in its lowest terms, i.e. $\frac{s'}{r'}$ where

$$d := \gcd(s,r) \text{ and } r' = r'(s,r) := \frac{r}{d}$$

The return value r' of the algorithm is the quantity of interest, and the hope is that $r' = r$ or that r can be recovered from r' somehow. (We also have $s' := \frac{s}{d}$ here, but the value of s' will be of no relevance.)

There are two cases in which the algorithm cannot guarantee successful identification of r:

(i) r can be odd;

(ii) If $d > 1$, then $r' < r$.

It is therefore useful to know what the probability is that the r' we recover as an estimate of r is correct, which will happen if and only if s and r are coprime. Given a and r (though r is unknown), it follows that the probability of obtaining $r' = k$ as output is

$$p_r(r' = k) = \frac{1}{r}\sum_{s=0}^{r-1}\delta_{k,\,r/\gcd(s,r)}. \tag{6}$$

In both of the unsuccessful cases highlighted above, the simplest solution would be to start afresh at step 1 after choosing another random a. However, if r' is even and $a^{r'} \not\equiv 1 \pmod N$, there is an argument for examining multiples $2r'$, $3r'$, ... until the order r is found. A nice discussion of this can be found in [17], chapter 17.

If r is recovered from step 5 (with probability given by eq. (6)), and furthermore if it is even (with probability at least $\frac{3}{4}$), we look for factors of N as described in the number theoretic part. If we still do not get any factors of N, we repeat the process again starting from step 1. Some of these challenges are further discussed in appendix B.

III. Results

A. Implementation of QFT

We coded our own circuitry for the QFT for up to five qubits (as well as the inverse QFT (IQFT)) as shown in the diagrams provided in section II A. As an initial test of the implementation, we considered the previous example with the initial state $|00\ldots0\rangle = |0\rangle$. When applying our own code to this state for various numbers of qubits n, our findings correspond to the theoretically expected resultant vector state $\frac{1}{\sqrt{M}}\sum_{k=0}^{M-1}|k\rangle$, i.e. the maximally mixed state. This can be shown for up to six qubits in tables VII and VIII in appendix C.

In figures 7 and 8 in appendix C, we also compare the distribution of measurement outcomes of a 3-qubit system when using our code, Qiskit's built-in QFT method and the $\mathbb{F}_M$ matrices determined previously, which correspond to inverse DFT by correlation (IDFT)). The three distributions are roughly the same, showing small random variations between them as we might expect, but overall they clearly display the tendency towards a uniform distribution of measuring the resultant state after applying the QFT to $|00\ldots0\rangle$.

Next, to validate our QFT implementation in a more general case, we compared it to Qiskit's built-in QFT method and the IDFT by correlation calculation for randomly initialized n-qubit states. All three implementations of the Fourier Transform were applied, and a measurement was sampled. If the empirical distribution resulting from the measurement had an ℓ^∞-distance of less than 0.01 (i.e. our implementation agrees with the two other methods in all points up to a tolerance of 0.01), we consider it a validation. This was repeated for 100 different randomly sampled n-qubit states to obtain the validation rates shown in table II for the QFT. IQFT validation rates can be found in table III. We see that as the number of measurements increases, the validation rate approaches 100%, indicating that our code works.

Shots	**1 qubit**	**2 qubits**	**3 qubits**	**4 qubits**	**5 qubits**
4 096	0.64	0.44	0.35	0.38	0.38
16 384	0.95	0.91	0.91	0.95	0.91
65 536	1.00	1.00	1.00	1.00	1.00

TABLE II. Validation rate of our QFT implementation across different numbers of qubits and measurements, where a run is counted as a validation if the ℓ^∞-distance between the empirical distributions of our QFT code, Qiskit's QFT and the IDFT matrices is less than 0.01. Columns indicate the number of qubits used in the simulation; rows indicate the number of measurements done. Each cell shows the fraction of successfully validated runs.

Shots	1 qubit	2 qubits	3 qubits	4 qubits	5 qubits
4 096	0.68	0.52	0.33	0.42	0.42
16 384	0.98	0.93	0.89	0.93	0.93
65 536	1.00	1.00	1.00	1.00	1.00

TABLE III. Validation rate of our IQFT implementation across different numbers of qubits and measurements. Each cell contains the fraction of successfully validated runs for the given parameters.

To confirm the unitarity of the code, we wish to see that when applying the IQFT after the QFT, we return to the state we started with, since $\mathrm{QFT}^\dagger\mathrm{QFT} = \mathbb{I}$. This test can be seen in fig. 3 below, where repeated measurement outcomes of a randomly initialized vector $|\psi\rangle$ are shown alongside the measurements of $\mathrm{QFT}^\dagger\mathrm{QFT}\,|\psi\rangle$. The results align with what we would expect from unitary operations, with $\mathrm{QFT}^\dagger\mathrm{QFT}$ corresponding to the identity operation.

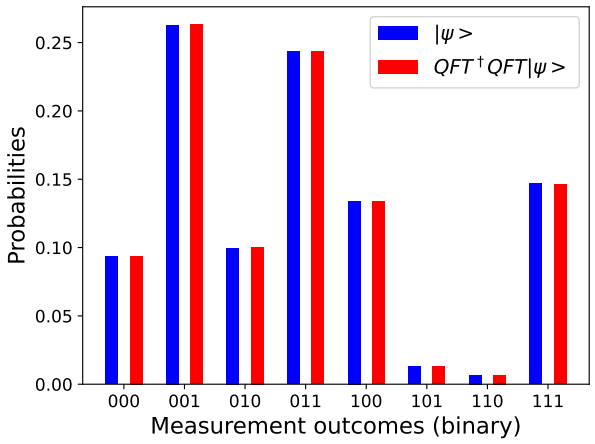

FIG. 3. Distribution of 2^{20} measurements of a randomly initialized vector $|\psi\rangle$ and $\mathrm{QFT}^\dagger\mathrm{QFT}\,|\psi\rangle$. The distributions are identical, showing the unitarity of the operations we implemented.

B. Implementation of Phase Estimation

Next, we implemented the phase estimation algorithm which utilizes the IQFT. To test our results, we fixed a phase $\frac{2\pi}{7}$, applying the following $CU1$ gate in the QPE circuit:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{\frac{2\pi i}{7}} \end{bmatrix}.$$

The algorithm will return the phase without the factor 2π, so we expect it to output $\frac{1}{7}$, or $0.\overline{001}$ in binary (with the bar signifying repeating digits). In fig. 4, we see the estimated phase after 1024 measurements of four qubits, meaning we are able to achieve four digits of precision. As we can see, the measurements are centered around the binary number 0.0010, the closest to the exact value $0.\overline{001}$ that we can achieve with four qubits. This aligns with our theoretical expectation.

In table IV, we show the estimated phase of an n-qubit system, for up to 10 qubits. Here, we display the mode measurement after 1024 measurements. As we can see, our implementation estimates the phase identically to the Qiskit implementation. We also achieve the same degree of precision as number of qubits used (given by n), meaning the mode measurement aligns with the closest possible representation of $\frac{1}{7}$ with n digits.

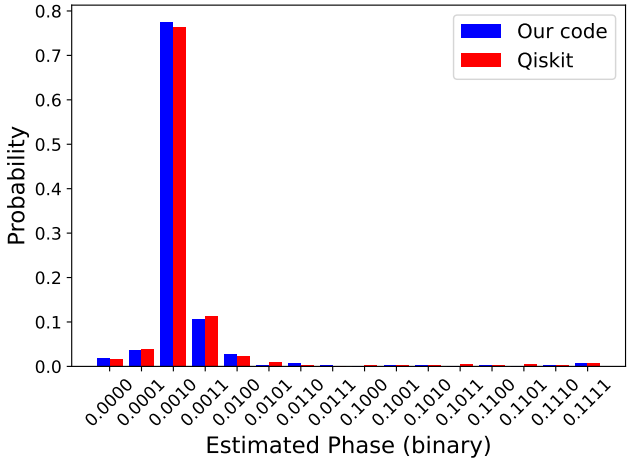

FIG. 4. Distribution of 1024 measurement results of a 4-qubit system after applying QPE with the $CU1$ gate as described. We expect the mode measurement to be as close as possible to the true phase $\frac{2\pi}{7}$ without the factor 2π, which corresponds to what we see here, with 0.0010 being the mode measurement.

Precision (digits)	Our code	Qiskit
1	0.0	0.0
2	0.01	0.01
3	0.001	0.001
4	0.0010	0.0010
5	0.00101	0.00101
6	0.001001	0.001001
7	0.0010010	0.0010010
8	0.00100101	0.00100101
9	0.001001001	0.001001001
10	0.0010010010	0.0010010010

TABLE IV. Mode of 1024 measurements of n-qubit systems after QPE is applied. For n qubits, we expect the value to be as close to $\frac{1}{7} = 0.\overline{001}$ as n digits can represent. We also see agreement between our implementation and Qiskit's built-in QPE method.

C. Shor's Algorithm

1. *Verifying the Order-Finding Algorithm*

The only part of Shor's algorithm that differs from the classical version of the factorization algorithm is the order-finding procedure. Hence, we take some time to verify that our quantum mechanical order finding procedure works.

For $N = 15$ and $a = 8$, the order is $r = 4$. We sample 1 000 estimates of r', given by the quantum order finding algorithm. We then compare with the distribution we expect, given by eq. (6). The compared distributions are shown in fig. 5. By visual inspection, the distributions appear to agree, indicating that our order-finding algorithm works as intended. We also verify that for each $a \neq 1$ which is coprime to $N = 15$, the empirical distribution of estimated orders (using 1 000 shots) agrees with the theoretical distribution given by eq. (6) up to an ℓ^∞-distance of 0.05.

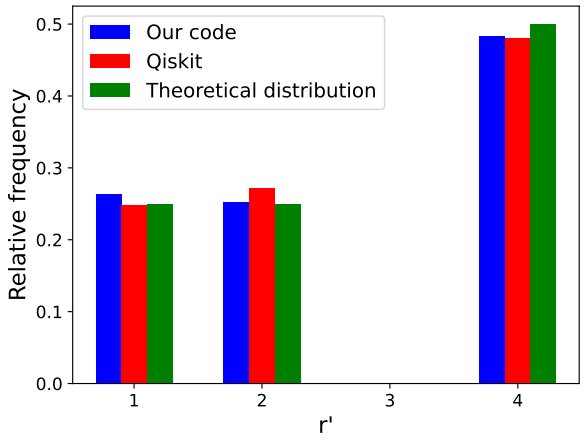

FIG. 5. Distribution of estimated r' using the quantum order finding algorithm for $N = 15$ and $a = 8$, using a 1 000-shot simulation with both our and Qiskit's QPE implementations. The theoretical distribution corresponds to the probabilities given by eq. (6).

2. *Running Full Algorithm*

With all the key components in place, we were finally able to run the full algorithm and see whether we would attain our goal of factoring an integer N, in particular $N = 15$ and $N = 21$. Our version of Shor's algorithm is coded completely from scratch, while the Qiskit version uses the same circuit setup that we have for all classical parts of the algorithm, with only the QPE component swapped for Qiskit's built-in QPE method. The results of our trials are shown in tables V and VI respectively.

	Our code	Qiskit
1 shot	11.31 s	0.58 s
20 shots	10.94 s	9.02 s

TABLE V. Runtime of our implementation of Shor's algorithm compared to Qiskit. Here, we factorized $N = 15$ and recorded the average time of a run over 20 runs for each implementation.

Our code	Qiskit
$5.55 \cdot 10^{-5}$ s	$6.89 \cdot 10^{-5}$ s
$1.17 \cdot 10^{-5}$ s	1 h 52 min 49.74 s
12 min 49.92 s	$3.53 \cdot 10^{-5}$ s
$1.03 \cdot 10^{-4}$ s	4 min 23.64 s
$4.34 \cdot 10^{-5}$ s	1 h 42 min 51.44 s

TABLE VI. Time taken to factorize $N = 21$ for five different runs of our implementation of Shor's algorithm compared to Qiskit. Clearly, there is massive variation between and within each of the methods. Some of this is explained by the randomness of the algorithm, though we do not quite understand what is causing the incredibly long runtime for some runs of the Qiskit code.

Based on our findings and the performance of the respective methods, it seems that Qiskit runs through the whole algorithm once for each shot, while our code samples all measurements from the probability distribution after only one run through all the gates. As a result, when using our code, the difference in time between one or 20 measurements is negligible. This scaling is clearly worse using Qiskit. However, their approach does make sense if we consider how this scaling might work on a real quantum computer, where we would need to run the full circuit for each measurement.

Factorizing 15 with our own code produced relatively consistent and clear results. There is some variation that is explained by the algorithm randomly picking an a which has a common factor with N. In this case, the algorithm trivially terminates with a classical calculation of $\gcd(a, N)$. When it came to factorizing 21 however, we ran into unexplainable phenomena. As can be seen in table VI, Qiskit sometimes takes an extremely long time to run just a single measurement shot, several orders of magnitude higher than the best-performing shots. While certain probabilistic elements of the algorithm may produce varying runtimes (in particular due to choice of a), we do not believe this can explain our findings here. In fact, all the time was spent on one run of the QPE algorithm. One hypothesis we have is that Qiskit may be spending time converting input operators into circuitry, but we have unfortunately not had the opportunity to verify or disprove this possible explanation.

IV. Discussion and Conclusion

This project focuses on highly relevant quantum algorithms with a large potential to disrupt key cryptographic systems that are in widespread use today. As such, we believe it is important to have a thorough understanding of the algorithms themselves and the underlying principles and components that could make them so powerful. Starting from scratch, we implemented the QFT and QPE, and found that our methods performed well when compared to Qiskit's built-in methods. After having validated that the main building blocks functioned as intended, we took on the challenge of coding and simulating Shor's algorithm, and with it, we were able to successfully factorize the integers 15 and 21. Our code performed quite consistently; however, the time taken to run the Qiskit code was highly variant. We are not entirely sure what is causing this.

It is important to remember that our findings come from simulations of quantum machines on classical hardware, meaning the performance we see is not necessarily representative of runtime on a real quantum machine. Though there is clearly a huge potential impact of RSA encryption being broken largely due to the power of quantum computing, we must ensure that we consider the individual use cases to see if the quantum implementation is a worthwhile pursuit. For instance, in terms of asymptotic runtime, it is clear that the QFT is superior to both DFT by correlation and FFT. However, this alone does not provide the full picture. For one thing, Big O notation does not account for the significant constant factor of the QFT. Additionally, the FFT returns the whole frequency spectrum in one evaluation, while the QFT returns one frequency per measurement. The FFT is thus suited for a wide range of use cases, from audio and image processing to solving differential equations, to name a few; the QFT, on the other hand, may be more appropriate when we are interested in one frequency only, such as in Shor's algorithm. While the probabilistic nature of measurement means we would seldom run a quantum algorithm a single time to obtain a result anyway, extracting essentially the same information from the QFT and FFT would require repeated measurements to determine the full frequency spectrum (up to some tolerance). In short, despite its asymptotic runtime, the QFT is unsuited for applications where we would like to avoid these repetitions. These aspects must be considered holistically to determine how we should perform computations as we enter a new era of realizable quantum technology.

All this is to say that the field of quantum computing is nascent and complex, and developments such as the Qiskit package allow for interesting explorations of the central quantum algorithms that we have explored. For further study on this topic, it would be useful to fully understand the timing issues we ran into with Qiskit when factorizing $N = 21$. We view this as a natural future extension of this project, as well as testing out IBM's machines with real quantum hardware [6] and exploring how the algorithms we have developed in this project fare in the real world.

[1] Various authors. *Qiskit Textbook*. Github, 2023.

[2] Brilliant contributors. RSA Encryption — Brilliant.org wiki. https://brilliant.org/wiki/rsa-encryption/, 2025. [Online; accessed 23-April-2025].

[3] David S. Dummit and Richard M. Foote. *Abstract Algebra*. John Wiley & Sons, USA, 3rd edition, 2004.

[4] G. H. Hardy and E. M. Wright. *An Introduction to the Theory of Numbers*. Oxford University Press, New York, 5th edition, 1979.

[5] Robert Hundt. *Quantum Computing for Programmers*. Cambridge University Press, 2022.

[6] IBM. Technology for the quantum future. https://www.ibm.com/quantum/technology, 2025.

[7] Ali Javadi-Abhari, Michael Treinish, Kevin Krsulich, Christopher J. Wood, James Lishman, Julien Gacon, Sebastien Martiel, Paul Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. Quantum computing with qiskit. *arXiv preprint*, arXiv:2405.08810, May 15 2024.

[8] Florian Luca and Igor E. Shparlinski. Average multiplicative orders of elements modulo n. *Acta Arithmetica*, 109(4):387–411, 2003.

[9] Wolfgang Scherer. *Mathematics of Quantum Computing: An Introduction*. Springer, Cham, Switzerland, 1st edition, 2019.

[10] David Schneider. A faster fast fourier transform. *IEEE Spectrum*, 49(3):12–13, 2012.

[11] P.W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134, 1994.

[12] Steven W. Smith. *The scientist and engineer's guide to digital signal processing*. California Technical Publishing, USA, 1997.

[13] Weisstein, Eric W. Modulo multiplication group — MathWorld–a wolfram web resource. https://mathworld.wolfram.com/ModuloMultiplicationGroup.html, 2025. [Online; accessed 23-April-2025].

[14] Wikipedia contributors. Continued fraction — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Continued_fraction, 2025. [Online; accessed 23-April-2025].

[15] Wikipedia contributors. Lagrange's theorem (group theory) — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Lagrange%27s_theorem_(group_theory), 2025. [Online; accessed 23-April-2025].

[16] Wikipedia contributors. Quantum fourier transform — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Quantum_Fourier_transform&oldid=1277591857, 2025. [Online; accessed 23-May-2025].

[17] Peter Young. An undergraduate course on quantum computing. https://bpb-us-e1.wpmucdn.com/sites.ucsc.edu/dist/7/1905/files/2025/03/phys_150_all.pdf, 2024. Fourth Edition.

A. Geometric Proof of Exponential Sums

Figure 6 shows a geometric proof of the identity in eq. (1).

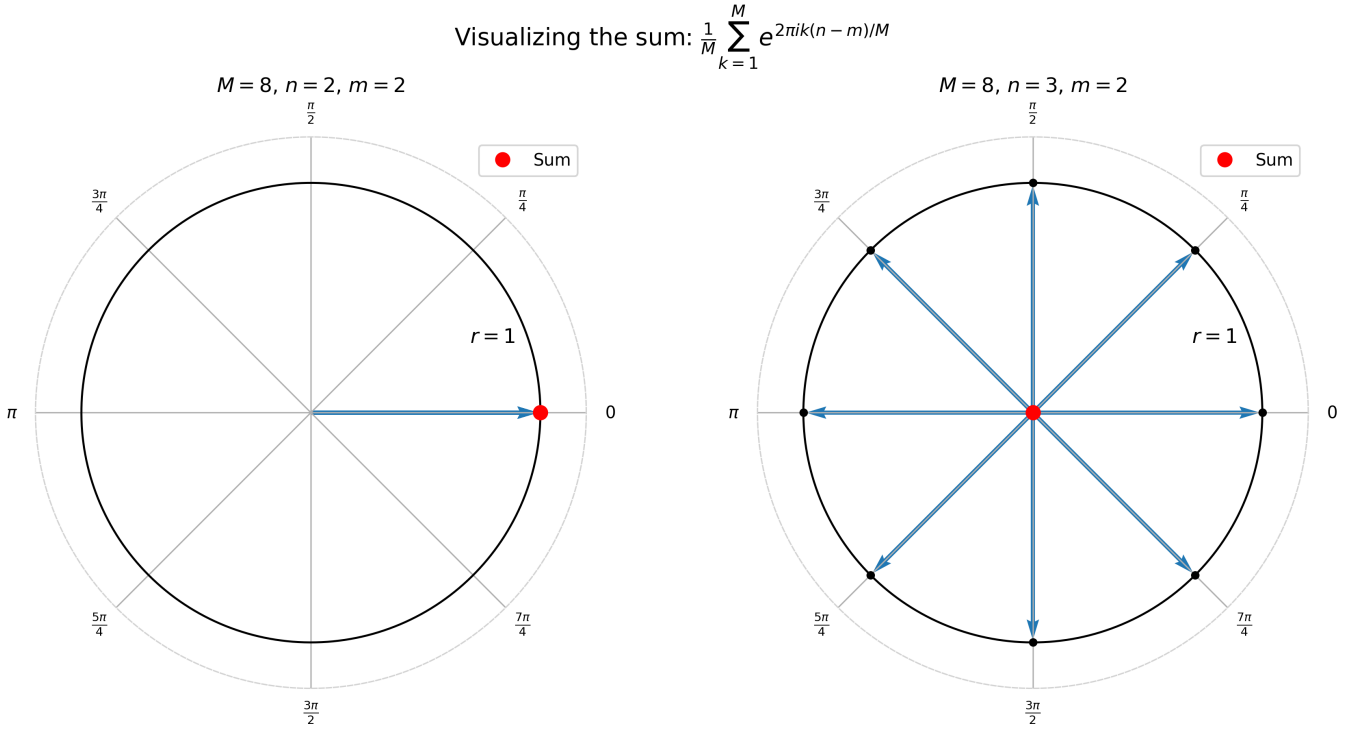

FIG. 6. Geometric proof.

If $n \equiv m \pmod M$, all the vectors align and the average of the sum of the vectors is 1. If $n \not\equiv m \pmod M$, all points are evenly spaced on the unit circle. Their mass center, which corresponds to the desired sum, is 0.

B. Challenges Related to Shor's Algorithm

Shor's algorithm is probabilistic in nature. In addition to having a bound on the probability that r is even, we also need some information on the probability that $\gcd(s, r) > 1$.

For $s \in [0, r-1]$, there are $\varphi(r)$ values of s that are coprime to r. Hence the sought probability is given by $\frac{\varphi(r)}{r}$.

The case of interest is r even. Then at least half of the s vil not be coprime to r. We would like a lower bound on the probability. Such a bound can be found for instance in Theorem 6.9 in [9] but stated in terms of the inverse fraction: For $r \geq 3$ the following inequality holds:

$$\frac{r}{\phi(r)} < \exp(\gamma) \log\log r + \frac{2.50637}{\log\log r}, \tag{B1}$$

where $\gamma := 0.5772156649\ldots$ denotes Euler's constant.

It is interesting to note that Shor in [11] similarly gives a Big-O bound $\mathcal{O}(\log\log r)$, citing the quite old number theory classic [4]. For our purposes, eq. (B1) alone is of limited value, since we a priori do not know the size of r given N. If $u(N)$ denotes the average order of elements in $U(\mathbb{Z}/N\mathbb{Z})$, a partial result of some interest is Theorem 6, (ii) in [8]: For any $\varepsilon > 0$,

$$u(N) > (\log N)^{(1/2-\varepsilon)\log_3 N} \tag{B2}$$

for all sufficiently large N.

As of 2016 [2], 1024-bit (309 digits) keys are considered risky, and most newly generated keys are 4096-bit (1234 digits). For $N \approx 10^{1234}$, and discarding the ϵ, eq. (B2) becomes $u(N) > 3.67 \cdot 10^6$. This is of a quite different order than N itself, though it must be noted that this just

concerns the average order. However, it gives some hope that it will be possible to give a meaningful bound on the probability of success for recovering r. This does indeed turn out to be the case, and is answered by Theorem 6.8 in [9]. The theorem and its formulation is quite technical; we will just provide the gist of it below.

Let $L = \lfloor 2\log_2 N \rfloor + 1$. There exists a quantum mechanical algorithm with which we can find the period r with a probability of at least

$$\frac{1}{10\log L} \tag{B3}$$

Had the denominator in eq. (B3) involved a linear term L, we would have been in serious trouble, as the probability of success would for all practical purposes be zero. However, the denominator is logarithmic, and this makes all the difference. In case N is of the order described earlier, eq. (B3) becomes 3.52×10^{-5}. Without any tricks to increase the probability of success, this means that one can expect to have to run the experiment 10^5 times to obtain a successful outcome.

C. QFT on Initial State Vector

The results provided in figures 7 and 8 are based on the QFT and IQFT on the initial state $|000\rangle$ (i.e. the 3-qubit system); tables VII and VIII go up to six qubits. We saw the same uniform measurement distribution for any number of qubits that we tested, confirming the expected state that we derived analytically in section II A.

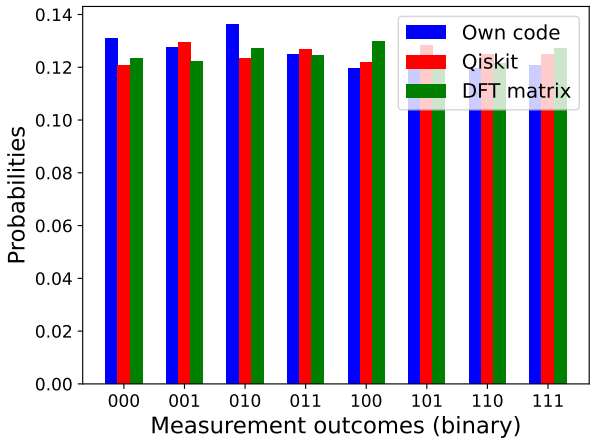

FIG. 7. Measurement distribution of QFT of $|000\rangle$.

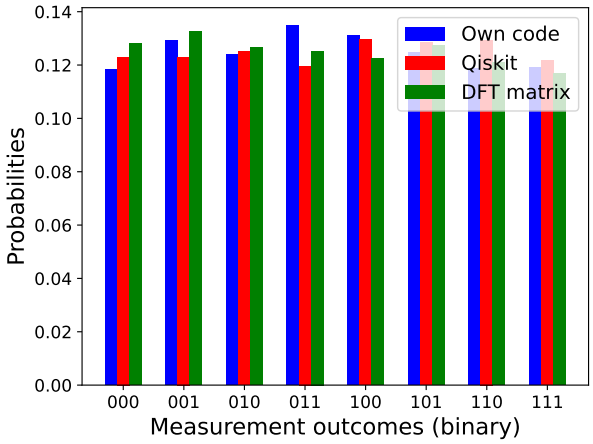

FIG. 8. Measurement distribution of IQFT of $|000\rangle$.

TABLE VII. State vector information for QFT and IQFT for inputs $|0\rangle$, $|00\rangle$ and $|000\rangle$. Only the first state is added for each qubit because the values are equal across all the states. The same values are obtained for our own code, creating quantum circuits using Qiskit and Qiskit's built-in QFT and IQFT methods.

State	Amplitude	Probability	Phase
	1-Qubit QFT		
$\|0\rangle$	$0.70710678 + 0.00000000j$	0.5000	0.00
	1-Qubit IQFT		
$\|0\rangle$	$0.70710678 + 0.00000000j$	0.5000	0.00
	2-Qubit QFT		
$\|00\rangle$	$0.50000000 + 0.00000000j$	0.2500	0.00
	2-Qubit IQFT		
$\|00\rangle$	$0.50000000 + 0.00000000j$	0.2500	0.00
	3-Qubit QFT		
$\|000\rangle$	$0.35355339 + 0.00000000j$	0.1250	0.00
	3-Qubit IQFT		
$\|000\rangle$	$0.35355339 + 0.00000000j$	0.1250	0.00

TABLE VIII. State vector information for QFT with initial states $|0000\rangle$, $|00000\rangle$ and $|00000\rangle$. The amplitude of all states is identical, so we show it for only one state.

State	**Amplitude**	**Probability**	**Phase**
	4-Qubit QFT		
$\|0000\rangle$	$0.2500 + 0.0000j$	0.0625	0.00
Manual (N=16)	0.2500		
	5-Qubit QFT		
$\|00000\rangle$	$0.1768 + 0.0000j$	0.0313	0.00
Manual (N=32)	0.1768		
	6-Qubit QFT		
$\|000000\rangle$	$0.1250 + 0.0000j$	0.0156	0.00
Manual (N=64)	0.1250		